CYBER SECURITY | CLOUD SECURITY

Python-Based Data Loss Prevention System for Cloud

The complete beginner-friendly explanation guide

Project meaning | Every command explained | Every keyword defined

What is inside

1 The big picture in plain words

2 How the system works (diagram + folders)

3 A real file traced step by step

4 Milestone 2 code, command by command

5 Milestone 3 code, module by module

6 The 7 detection patterns, with tests

7 Real output from running the project

8 Glossary of every keyword

9 Milestone checklist, limitations, viva Q&A

10 One-page cheat sheet

1. The big picture, in plain words

Think of it like airport security
Before anyone boards a plane, their bag goes through a scanner. Harmless bags pass, suspicious bags are pulled aside for a manual check, and dangerous items are confiscated. Your project is that scanner, but for **files** that are about to be stored in the cloud.

What does the title mean?

Words in the title	Meaning
Python-Based	The whole system is written in the Python programming language.
Data Loss Prevention (DLP)	A security method that finds sensitive data and stops it from leaking or being stored where it should not be.
System	Several small parts (modules) working together as one pipeline.
for Cloud	The files are headed to cloud storage such as AWS S3, a place on the internet that many people can reach, so mistakes there are costly.

The problem

- Employees upload files to the cloud every day. Some files accidentally contain **passwords, API keys, card numbers or ID numbers**.
- If such a file lands in a public or poorly protected bucket, attackers can find and abuse it. This is a **data breach**.
- Reading every file by hand is slow and error-prone, so the check must be **automatic**.
- Organisations also need **proof** of what was checked and decided, which is the audit log.

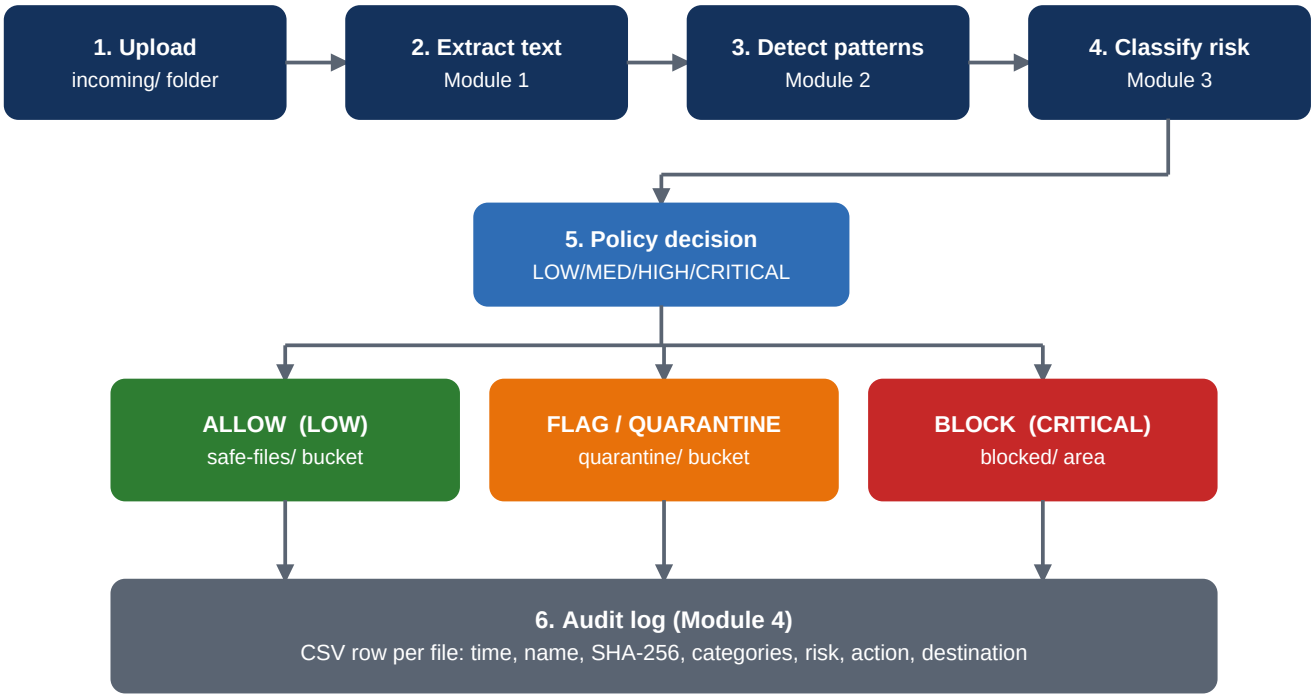
What the prototype does

In one sentence
It **reads** a file, **searches** it for sensitive patterns, gives it a **risk level**, applies a **policy** (allow, flag, quarantine or block), **stores** it in the matching place, and **records** the decision in a log.

Why local folders instead of real AWS?

A real S3 bucket needs an AWS account, credentials and sometimes cost. To keep the project safe and free, **folders on disk act as buckets**: `safe-files/`, `quarantine/` and `blocked/`. The decision logic is exactly what a real deployment would use. Only the final storage step would change, from copying to a folder to uploading with `boto3`.

2. How the system works



Read the diagram top to bottom. Every file takes the same path, and only the **risk level** decides which of the three exits it uses.

Folder layout created by the notebook

```
dlp_cloud_simulation/
|-- incoming/ files waiting to be scanned (the 'upload' area)
|-- safe-files/ clean files (ALLOW)
|-- quarantine/ files held for review (FLAG and QUARANTINE)
|-- blocked/ dangerous files (BLOCK)
`-- audit/
    `-- dlp_audit_log.csv one row per scanned file
```

The decision table

Risk	Triggered by	Action	Goes to	Meaning
LOW	Nothing sensitive found	ALLOW	safe-files/	Passes normally
MEDIUM	Email address, phone number	FLAG	quarantine/	Personal data; needs a look
HIGH	PAN-like or Aadhaar-like ID numbers	QUARANTINE	quarantine/	Identity data; isolate for review
CRITICAL	Password, API key, card number	BLOCK	blocked/	Direct security risk; stop it

Key rule: highest risk wins
If a file contains both an email (MEDIUM) and a password (CRITICAL), the file is CRITICAL and gets BLOCKED. One dangerous item is enough.

3. One real file, traced step by step

Follow `credentials_demo.txt` through every module. All values below come from actually running the project logic.

1	**Step 1: Arrives in incoming/**
	The notebook wrote this file into `incoming/`. Contents:
	`Demo configuration`
	`password = DemoPassword123!`
	`api_key = DEMO_API_KEY_123456789`

2	**Step 2: Module 1 reads the text** The extension is .txt, so `read_text()` returns the three lines above as one string.
3	**Step 3: Module 2 detects patterns** All 7 regex rules are tried. Two match: `{'Password-like assignment': 1, 'API-key-like value': 1}` Only the **counts** are kept, never the secret values.
4	**Step 4: Module 3 classifies risk** Both categories are CRITICAL, so the file risk is **CRITICAL**.
5	**Step 5: Policy decision** CRITICAL maps to **BLOCK**.
6	**Step 6: Module 4 routes the file** The file is copied to `dlp_cloud_simulation/blocked/credentials_demo.txt`.
7	**Step 7: Audit record written** A row is appended to the CSV with the time, file name, hash `094abe149f12dc66...`, categories, counts, risk=CRITICAL, action=BLOCK and the destination path.

Why log a hash instead of the file?
A SHA-256 hash is a 64-character fingerprint. It identifies the exact file and proves it was not altered, without copying the sensitive content into the log. Logging the matched values would turn the log itself into a leak.

4. Milestone 2 code, command by command

Goal of Milestone 2
Prepare the workspace and identify what the project needs: Python, libraries, simulated cloud resources and a dataset. Nothing is scanned yet.

4.1 Check the Python version

Code	What it does, in simple words
`import sys`	Brings in **sys**, a built-in toolbox with facts about the running Python.
`print("Python Version:")`	Prints a label on screen.
`print(sys.version)`	Prints the exact Python version so you know the environment (for example 3.12.x).

4.2 Install the libraries

Code	What it does, in simple words
`!pip install pandas numpy pypdf python-docx openpyxl boto3 matplotlib`	The **!** means 'run this in the computer's shell, not as Python'. **pip** is Python's app store; **install** downloads each library. Seven libraries are fetched: pandas (tables), numpy (maths), pypdf (read PDFs), python-docx (read Word files), openpyxl (Excel), boto3 (AWS) and matplotlib (charts).

4.3 Import the libraries

Code	What it does, in simple words
`import pandas as pd`	Loads pandas and nicknames it **pd** so you type less.
`import numpy as np`	Loads NumPy with nickname **np**.
`import re`	Loads the regular-expression (pattern search) module.
`import os / import json`	**os** talks to the operating system; **json** reads/writes JSON text.
`import boto3`	Loads the AWS toolkit.
`import matplotlib.pyplot as plt`	Loads the chart-drawing part of matplotlib as **plt**.
`from pypdf import PdfReader`	Takes only the **PdfReader** tool from pypdf.
`from docx import Document`	Takes only the **Document** tool from python-docx.
`print("All required libraries imported successfully.")`	Runs only if every import above worked, so it doubles as a success message.

4.4 Verify the environment

Code	What it does, in simple words
`import sys, pandas, numpy, boto3, matplotlib`	Loads each library again so its version number can be read.
`print("Pandas:", pandas.__version__)`	Every library stores its version in **__version__**. The same line pattern is repeated for NumPy, Boto3 and Matplotlib.
`sys.version.split()[0]`	**split()** cuts the long version text at spaces, and **[0]** keeps the first piece (just the number).
`print("\nEnvironment setup completed successfully.")`	**\n** means 'start a new line', which leaves a blank line before the message.

4.5 Architecture and storage notes

These two cells hold only text in triple quotes. They run no logic. They **document the design**: the flow User, File Upload, Python DLP Scanner, Detection, Policy Evaluation, then ALLOW / QUARANTINE / BLOCK into three storage areas (safe-files, quarantine, blocked). Documenting resources is part of 'Resource Identification'.

4.6 Build the synthetic dataset

Why fake data?
Using real people's data for testing would itself be a privacy risk. Synthetic data looks real but belongs to nobody, which also follows the **data minimization** principle.

Code	What it does, in simple words
`data = { "Category": [...], "Data_Type": [...], "Sample_Value": [...] }`	A **dictionary** (labelled lists). Each of the three keys holds 10 items. Row 1 is Personal Information / Name / Rahul Sharma, row 2 is Email, and so on, covering Personal, Financial, Identity, Credentials and Normal data.
`dataset = pd.DataFrame(data)`	Turns the dictionary into a pandas table (a **DataFrame**) with 3 columns and 10 rows.
`dataset`	Putting a variable name alone on the last line makes Colab display it as a table.
`dataset.to_csv("dlp_test_dataset.csv", index=False)`	Saves the table as a CSV file. **index=False** avoids an extra 0,1,2... numbering column.
`print("Number of records:", len(dataset))`	**len()** counts the rows. Output: 10.

Dataset at a glance

Category	Types	Examples (all fake)
Personal Information	Name, email, phone	Rahul Sharma, rahul@example.com, 9876543210
Financial Information	Card number, bank account	4111111111111111 (a standard test card number)
Identity Information	PAN-like, Aadhaar-like	ABCDE1234F, 123456789012
Credentials	Password, API key	DemoPassword123, DEMO_API_KEY_123456
Normal Data	Harmless text	This is a normal non-sensitive document.

5. Milestone 3 code, module by module

Goal of Milestone 3
Build the working modules and show security and compliance ideas in action: detection, risk classification, policy enforcement, isolation and auditing.

5.1 Setup: simulated cloud and test files

Code	What it does, in simple words
`from pathlib import Path`	**Path** treats file and folder locations as objects that are easy to join and use.
`BASE = Path("dlp_cloud_simulation")`	The main folder name. CAPITAL names are a habit meaning 'a fixed setting'.
`INCOMING = BASE / "incoming"`	The **/** joins path pieces. SAFE, QUARANTINE, BLOCKED and AUDIT are created the same way.

Code	What it does, in simple words
`for folder in [INCOMING, SAFE, ...]: folder.mkdir(parents=True, exist_ok=True)`	Loops over all five folders and creates each one. **parents=True** also creates missing parent folders. **exist_ok=True** means no error if it already exists, so the cell can be re-run.
`sample_files = {"normal_report.txt": "...", ...}`	A dictionary of five fake files. Key = file name, value = file text. Two are clean-ish (normal, contact), three carry riskier data (identity, credentials, payment).
`for name, content in sample_files.items(): (INCOMING / name).write_text(content, encoding="utf-8")`	For each pair, writes a real file into incoming/. **utf-8** is the text encoding, so characters are stored correctly.
`print(f"Created {len(sample_files)} synthetic test files.")`	An **f-string**: the part in { } is replaced by a value. Prints 'Created 5 ...'.
`INCOMING.resolve()`	Shows the full absolute path of the folder on disk.
`sorted(p.name for p in INCOMING.iterdir())`	**iterdir()** lists items in the folder, **p.name** takes each file name, **sorted** puts them in alphabetical order.

5.2 Module 1: File reading and text extraction

INPUT	**OUTPUT**
A file path such as `report.pdf`	The plain text inside the file

Code	What it does, in simple words
`def extract_text(file_path):`	Defines a reusable function named extract_text.
`path = Path(file_path)`	Converts the path text into a Path object.
`ext = path.suffix.lower()`	Finds the extension (.PDF) and makes it lower case (.pdf) so capital letters do not matter.
`if ext in {".txt", ".csv", ".json", ".log"}: return path.read_text(...)`	Plain-text formats are just read directly. **errors="replace"** swaps unreadable characters instead of crashing.
`if ext == ".pdf": PdfReader(str(path)).pages`	Opens the PDF and loops over its pages.
`page.extract_text() or ""`	Gets a page's text. If the page has none (for example a scanned picture) **or ""** uses empty text instead of None.
`"\n".join(...)`	Joins all page texts, placing a new line between them.
`if ext == ".docx": doc = Document(str(path))`	Opens a Word document.
`parts = [p.text for p in doc.paragraphs]`	A **list comprehension**: a one-line loop that collects the text of every paragraph.
`for table in doc.tables: ... " | ".join(cell.text for cell in row.cells)`	Also reads text hidden inside Word tables, separating cells with ' | '.
`raise ValueError(f"Unsupported file type: {ext}")`	Unknown formats stop with an error. This is a deliberate safe design: a file that cannot be read is never silently treated as safe.

5.3 Module 2: Sensitive-data detection

INPUT Text from Module 1	**OUTPUT** A dictionary of categories and counts, for example `{'Email address': 1}`

Code	What it does, in simple words
`PATTERNS = {"Email address": r"...", ...}`	A dictionary: category name on the left, its regex pattern on the right. The **r** before the quote makes a **raw string** so backslashes are kept exactly as typed (see Section 6).
`def detect_sensitive_data(text):`	Function that takes text and returns what it found.
`findings = {}`	Starts with an empty results box.
`for category, pattern in PATTERNS.items():`	Tries every pattern, one after another.
`matches = re.findall(pattern, text)`	**re.findall** returns a list of everything in the text that fits the pattern.
`if matches:` `  findings[category] = len(matches)`	If anything matched, store **how many**. Only the count is saved, never the value, so secrets do not spread.
`return findings`	Hands the results back to the caller.

5.4 Module 3: Risk classification and policy

INPUT The findings dictionary	**OUTPUT** A risk level (LOW, MEDIUM, HIGH or CRITICAL) and then an action

Code	What it does, in simple words
`CATEGORY_RISK = {"Password-like assignment": "CRITICAL", ...}`	A severity chart. Password, API key and card = CRITICAL. Aadhaar and PAN = HIGH. Email and phone = MEDIUM.
`RISK_ORDER = {"LOW":0, "MEDIUM":1, "HIGH":2, "CRITICAL":3}`	Gives each level a number so Python can tell which is higher.

`def classify_risk(findings):`
`  if not findings:`
`  return "LOW"`
	Nothing found means LOW.

`levels = [CATEGORY_RISK.get(c, "MEDIUM") for c in findings]`	Looks up each found category's severity. **.get(c, "MEDIUM")** falls back to MEDIUM if a category is missing from the chart.
`return max(levels, key=lambda level: RISK_ORDER[level])`	Picks the **highest** level. **lambda** is a tiny nameless function that tells max() to compare using the numbers in RISK_ORDER.
`def apply_policy(risk):` `  return {...}[risk]`	A dictionary used like a lookup table: LOW to ALLOW, MEDIUM to FLAG, HIGH to QUARANTINE, CRITICAL to BLOCK.
`for level in [...]:` `  print(level, "->", apply_policy(level))`	Prints all four mappings so the policy is visible in the output.

5.5 Module 4: Storage routing and audit logging

INPUT	OUTPUT
A file in incoming/	The file copied to the right folder plus a new row in the audit log

Code	What it does, in simple words
`import shutil, hashlib, json` `from datetime import datetime, timezone`	Tools for copying files (shutil), hashing (hashlib), JSON text (json) and timestamps (datetime).
`AUDIT_FILE = AUDIT / "dlp_audit_log.csv"`	Where the log will be saved.
`def sha256_file(file_path):` `  h = hashlib.sha256()`	Creates a SHA-256 fingerprint calculator.
`with open(file_path, "rb") as f:`	Opens the file in **binary** mode (rb = read bytes). **with** closes it automatically afterwards.
`for chunk in iter(lambda: f.read(8192), b""):` `  h.update(chunk)`	Reads the file in 8 KB pieces until nothing is left and feeds each piece to the calculator. Works even for huge files without using much memory.
`return h.hexdigest()`	Returns the finished 64-character fingerprint as text.
`def process_file(file_path):`	The full pipeline for ONE file.

`content = extract_text(path)`
`findings = detect_sensitive_data(content)`
`risk = classify_risk(findings)`
`action = apply_policy(risk)`
	Modules 1, 2 and 3 run in order: read, detect, classify, decide.

`destination_dir = {"ALLOW": SAFE, "FLAG": QUARANTINE,` `  "QUARANTINE": QUARANTINE, "BLOCK": BLOCKED}[action]`	Picks the target folder from the decision.
`shutil.copy2(path, destination)`	Copies the file and keeps its original timestamps. It **copies** (does not move), so the original stays in incoming/.
`event = {"timestamp_utc": datetime.now(timezone.utc).isoformat(), ...}`	Builds one audit record: UTC time, file name, SHA-256, categories, counts, risk, action, destination.
`json.dumps(findings, sort_keys=True)`	Converts the counts dictionary into text so it fits in a single CSV cell.
`row = pd.DataFrame([event])`	Makes a one-row table from the record.
`if AUDIT_FILE.exists():` `  row = pd.concat([pd.read_csv(AUDIT_FILE), row], ignore_index=True)`	If a log already exists, read it and add the new row at the bottom. **ignore_index=True** renumbers the rows neatly.
`row.to_csv(AUDIT_FILE, index=False)`	Saves the log to disk.
`return {"File": ..., "Risk": ..., "Decision": ...}`	Returns a short summary used for the results table on screen.
`results = [process_file(p) for p in sorted(INCOMING.iterdir()) if p.is_file()]`	Runs process_file on every file in incoming/ (skipping sub-folders) and collects the summaries.
`results_df = pd.DataFrame(results)` `display(results_df)`	Turns the summaries into a table and shows it. **display()** is a notebook function that draws tables nicely.

5.6 Audit review, compliance notes and S3 readiness

Code	What it does, in simple words
`audit_df = pd.read_csv(AUDIT_FILE)`	Reads the saved log back into a table.
`display(audit_df[["timestamp_utc", ...]])`	Double square brackets pick only the listed columns, so the display stays readable.
`print("Files processed:", len(results_df))`	Prints how many files were handled (5).
`try: ... except Exception as exc:`	**Error handling**. Python tries the code in **try**. If anything fails, **except** runs instead of crashing the notebook.
`session = boto3.Session()`	Creates an AWS session, which automatically looks for configured credentials.
`if session.get_credentials() is None:`	True when no AWS credentials exist. The notebook then says it will keep using the local simulation. It never uploads or changes anything in S3.
`type(exc).__name__`	Prints only the error's type name, not a long message.

Security ideas shown in the notebook (Section 9 of Milestone 3)

Mechanism	How the project shows it
Data minimization	Only synthetic values are used.
Least privilege	A real AWS integration should use an IAM role with only the permissions it needs.
Credential protection	AWS keys must never be written into the notebook.
Isolation	Review-required files go to quarantine instead of normal storage.
Auditability	Hashes and decisions are logged without secret values.
Policy enforcement	The same risk always gives the same decision.

6. The 7 detection patterns, decoded and tested

A **regular expression (regex)** describes the *shape* of text. The tables below show what each symbol means and then test every rule on real examples.

Symbols used

Symbol	Meaning
`\b`	Word boundary: the match must begin or end at the edge of a word.
`[A-Za-z0-9._%+-]`	Any ONE character from this list.
`+`	One or more of the thing before it.
`{5} {4} {2,} {13,19}`	Exactly 5; exactly 4; 2 or more; between 13 and 19.
`\d`	One digit, 0 to 9.
`(?<!\d)`	'Not preceded by a digit.' Stops matching inside a longer number.
`(?!\d)`	'Not followed by a digit.'

Symbol	Meaning
`(?:...)`	A group used only for grouping.
`?`	The thing before it is optional.
`|`	OR.
`(?i)`	Ignore capital vs small letters.
`\s*`	Zero or more spaces.
`[^\s,;]+`	One or more characters that are NOT space, comma or semicolon.

Each rule with live tests

Rule	In plain English	Test text	Matches?
PAN-like	5 capital letters, 4 digits, 1 capital letter.	`ABCDE1234F`	YES
		`abcde1234f`	no
		`ABCD1234F`	no
Aadhaar-like	Exactly 12 digits that are not part of a longer number.	`123456789012`	YES
		`1234567890123`	no
		`12345678901`	no
Phone number	Optional +91, then a digit 6 to 9 followed by 9 digits. Indian mobiles start with 6-9.	`9876543210`	YES
		`1234567890`	no
		`+91 9876543210`	YES
Email address	Name characters, an @, domain, a dot, then 2 or more letters.	`rahul@example.com`	YES
		`rahul@com`	no
		`a.b+c@x.co.in`	YES
Card-like	13 to 19 digits, optionally split by single spaces or hyphens.	`4111 1111 1111 1111`	YES
		`4111111111111111`	YES
		`123456789012`	no
Password-like	The word password, passwd or pwd, then : or =, then the value.	`password = abc123`	YES
		`Password: xyz`	YES
		`passwords are important`	no
API-key-like	api_key / secret_key / access_token followed by : or =, OR the demo key format DEMO_API_KEY_...	`api_key = ABC123`	YES
		`DEMO_API_KEY_555`	YES
		`apikey is here`	no

Patterns only check the SHAPE
A random 12-digit number matches the Aadhaar rule, and any long number can match the card rule. These are **false positives**. A stronger system adds the Luhn checksum for cards and keyword context. Likewise, a secret written in an unusual style can be missed (a **false negative**).

7. Real output from running the project

This is what the results table and audit log contain after processing the five files (SHA-256 shortened to 16 characters for space).

Results table

File	Detected categories	Risk	Decision	Destination folder
normal_report.txt	None	**LOW**	**ALLOW**	safe-files/
contact_list.txt	Email address, Phone number (10 digits)	**MEDIUM**	**FLAG**	quarantine/
identity_record.txt	PAN-like value, Aadhaar-like 12-digit value	**HIGH**	**QUARANTINE**	quarantine/
credentials_demo.txt	Password-like assignment, API-key-like value	**CRITICAL**	**BLOCK**	blocked/
payment_demo.txt	Payment-card-like number	**CRITICAL**	**BLOCK**	blocked/

Audit log (dlp_audit_log.csv)

file_name	sha256 (first 16)	match_counts	risk	action
`normal_report.txt`	`d9fb331dd2071604`	`{}`	LOW	ALLOW
`contact_list.txt`	`7ebc1b3acc4066f7`	`{"Email address": 1, "Phone number (10 digits)": 1}`	MEDIUM	FLAG
`identity_record.txt`	`fdc76b37851e3b66`	`{"Aadhaar-like 12-digit value": 1, "PAN-like value": 1}`	HIGH	QUARANTINE
`credentials_demo.txt`	`094abe149f12dc66`	`{"API-key-like value": 1, "Password-like assignment": 1}`	CRITICAL	BLOCK
`payment_demo.txt`	`cdf8bddfab04704b`	`{"Payment-card-like number": 1}`	CRITICAL	BLOCK

The real log also stores `timestamp_utc`, the full 64-character hash, `detected_categories` and `destination`. The hash values are the same every run because the file contents do not change, but timestamps will differ.

Evidence to screenshot for your submission
(1) the environment verification output, (2) the results table, (3) the audit log table, (4) the folder tree showing files in safe-files, quarantine and blocked.

8. Glossary: every keyword defined

Keyword	Simple definition	Where in this project
Security and cloud ideas		
Data Loss Prevention (DLP)	Tools and rules that detect sensitive data and stop it from leaking or being stored in the wrong place.	The whole project.
Cloud computing	Renting storage and computers over the internet instead of owning hardware.	Target is cloud storage.
AWS	Amazon Web Services, Amazon's cloud platform.	Intended real platform.
S3 (Simple Storage Service)	AWS storage that keeps files as objects inside buckets.	Simulated by folders.
Bucket	A named container in S3 that holds files.	safe-files, quarantine, blocked.
IAM	Identity and Access Management: AWS controls for who may do what.	Mentioned for real deployment.
Least privilege	Giving only the minimum permissions needed.	Advice for AWS use.
Cyber security	Protecting systems and data from attack or theft.	Project domain.
Data breach	Confidential data exposed to people who should not see it.	What DLP tries to prevent.
Sensitive data	Information that causes harm if leaked.	What gets detected.
PII	Personally Identifiable Information: data that identifies a person.	Names, emails, phones.
Credentials	Secrets that prove identity: passwords, keys, tokens.	CRITICAL category.
API key / token	A secret string that lets a program use a service.	Detected by Module 2.
PAN	Indian tax ID: 5 letters, 4 digits, 1 letter.	ABCDE1234F format.
Aadhaar	India's 12-digit national ID number.	Any 12-digit value is flagged.
Payment card number	13 to 19 digit number on a bank card.	4111111111111111 test value.
Compliance	Following data-protection laws and standards (GDPR, DPDP Act, PCI-DSS).	Supported, not proven.
Data minimization	Use only the data you truly need.	Synthetic data.
Synthetic data	Fake data that looks real but belongs to nobody.	The whole dataset.
Policy enforcement	Automatically applying a rule such as allow or block.	Module 3 and 4.
Quarantine	An isolated holding area for suspicious files awaiting review.	quarantine/ folder.
Audit log	A record of events: what, when, decision.	dlp_audit_log.csv.
Risk classification	Labelling data LOW, MEDIUM, HIGH or CRITICAL.	Module 3.
Hash (SHA-256)	A one-way function giving a 64-character fingerprint of a file; any change alters it.	Logged for each file.
False positive	Harmless data wrongly flagged.	Random 12-digit number.
False negative	Sensitive data that is missed.	Unusual secret formats.
Prototype	An early working model, not production-ready.	This project.
Simulation	A simpler stand-in that imitates a real system.	Folders act as buckets.

Keyword	Simple definition	Where in this project
Programming and tools		
Python	General-purpose programming language.	Entire code.
Google Colab	Free browser notebook service that runs Python in the cloud.	Development environment.
Notebook (.ipynb)	A document that mixes code, text and results.	The submission file.
Library / package	Pre-written code you import and reuse.	pandas, boto3, etc.
pip	Python's installer for libraries.	!pip install ...
pandas	Library for working with tables of data.	Dataset and audit log.
NumPy	Library for fast numeric arrays.	Installed, not used.
Matplotlib	Library for drawing charts.	Installed, not yet used.
boto3	Official AWS SDK for Python.	Readiness check.
SDK	Software Development Kit: a library for using a service.	boto3 is one.
pypdf	Reads PDFs and extracts text.	Module 1.
python-docx	Reads Word (.docx) files.	Module 1.
openpyxl	Reads and writes Excel files.	Installed for later use.
Regular expression (regex)	A pattern language describing text shapes.	All 7 detection rules.
re module	Python's built-in regex tools.	re.findall.
pathlib / Path	Built-in way to handle file and folder paths.	Folder setup.
shutil	Built-in file operations such as copying.	shutil.copy2.
hashlib	Built-in module for hashes such as SHA-256.	sha256_file.
datetime / UTC	Date and time tools; UTC is a universal time standard.	Audit timestamp.
JSON	A text format of key-value data.	Counts in the log.
CSV	Comma-Separated Values: a plain text table.	Dataset and log.
DataFrame	pandas table with rows and labelled columns.	dataset, results_df.
Dictionary (dict)	Key-value pairs like {'Email address': 1}.	PATTERNS, findings, etc.
Function (def)	A named, reusable block of code.	extract_text, process_file.
Loop (for)	Repeats code for each item in a collection.	Folders, files, patterns.
List comprehension	A one-line loop that builds a list.	[p.text for p in ...]
lambda	A tiny one-line nameless function.	Used inside max().
f-string	Text with {values} filled in, written f"...".	Print messages.
try / except	Run risky code and handle failure without crashing.	S3 readiness check.
raw string r"..."	String where backslashes are literal.	Regex patterns.

9. Milestone checklist, limitations and viva Q&A;

Does it meet the milestones?

MS	Requirement	Status	Evidence / note
M2	Python environment set up	**Done**	Colab, Python version check, installs, verification.
M2	Packages and libraries identified	**Done**	Seven libraries installed and imported.
M2	Simulated cloud resources	**Partly**	Diagram and folder layout given; add a short written table of resources.
M2	Dataset identified	**Done**	Synthetic dataset, five categories, saved as CSV.
M2	Written description of requirements	**Add**	Add a short table: package, purpose, resource, dataset.
M3	Individual modules working	**Done**	Four modules, each demonstrable.
M3	Cloud security mechanisms	**Done**	Quarantine, blocking, least-privilege notes, audit log.
M3	Compliance-related controls	**Done**	Data minimization, auditability; honest non-claim of legal compliance.
M3	Output evidence	**Check**	Run all cells and save the notebook with outputs.

Honest limitations

- Regex checks only **shape**, so false positives and false negatives are possible.
- No real S3 is used. Folders simulate buckets.
- BLOCK still copies the file into a local *blocked* folder for demonstration. A real system would reject the upload before storage.
- No Luhn check for cards, no encryption, no user login, no OCR for scanned PDFs.
- `numpy`, `matplotlib` and `openpyxl` are installed but not used yet.
- The prototype supports good practice but does not prove compliance with any law or standard.

Likely viva questions

Question	Short answer
What is DLP?	A system that detects sensitive data and enforces rules to stop leakage or misuse.
Why simulate the cloud?	Real AWS needs credentials and may cost money. Folders show the same workflow safely.
Why synthetic data?	So no real person's information is exposed (data minimization).
How is risk decided?	The highest-severity category found in the file decides the risk.
Why store a hash?	It identifies the exact file and proves integrity without copying its content.
Why not log matched values?	The log would become a second place where secrets leak.
What happens to a file with an email and a password?	It becomes CRITICAL because the password is CRITICAL, so it is BLOCKED.
Why reject unknown file types?	An unreadable file must not be assumed safe.

Question	Short answer
How would you move to real S3?	Use boto3 with an IAM role of least privilege and upload to safe or quarantine buckets; reject BLOCK files before upload.
How could detection improve?	Luhn validation, context keywords, machine-learning classifiers and OCR.

10. One-page cheat sheet

Pipeline
Upload (incoming/) > **Extract** (Module 1) > **Detect** (Module 2) > **Classify** (Module 3) > **Policy** > **Route + Log** (Module 4)

Module	Function	Job
Module 1	extract_text()	Reads TXT, CSV, JSON, LOG, PDF, DOCX; rejects others.
Module 2	detect_sensitive_data()	Runs 7 regex rules; returns category counts only.
Module 3	classify_risk(), apply_policy()	Highest severity wins; LOW/MEDIUM/HIGH/CRITICAL to ALLOW/FLAG/QUARANTINE/BLOCK.
Module 4	process_file()	Routes file to a folder, hashes it, appends an audit row.

Risk	Action	Folder	Examples
LOW	**ALLOW**	safe-files/	Plain text
MEDIUM	**FLAG**	quarantine/	Email, phone
HIGH	**QUARANTINE**	quarantine/	PAN-like, Aadhaar-like
CRITICAL	**BLOCK**	blocked/	Password, API key, card number

Topic	Remember
Python basics	dict = labelled data, list = ordered items, def = function, for = loop, try/except = handle errors
Key libraries	pandas (tables), re (patterns), pathlib (paths), shutil (copy), hashlib (fingerprint), boto3 (AWS), pypdf and python-docx (read files)
Five test files	normal_report (LOW), contact_list (MEDIUM), identity_record (HIGH), credentials_demo (CRITICAL), payment_demo (CRITICAL)
Golden lines	Only counts are logged, never secrets. Unknown file types are rejected. Highest risk wins. This is a simulation and does not prove legal compliance.